Lab 3 – Spot and fix vulnerabilities

Goal - Strengthen security awareness in AI-generated code.

Scenario: Create a user management system with registration, login, password reset, and profile management - a system where security vulnerabilities commonly occur.

Phase 1: AI Code Generation

Prompt AI to create:

UserManagementSystem with:

1. registerUser(email, password, personalInfo)
2. authenticateUser(email, password)
3. resetPassword(email, newPassword, resetToken)
4. updateProfile(userId, profileData)
5. deleteUser(userId, confirmPassword)
6. getUserData(userId, requestingUserId)

Phase 2: Deliberate Vulnerability Introduction The instructor will provide pre-seeded vulnerable code snippets that demonstrate common OWASP Top 10 issues:

A01 - Broken Access Control

```
def getUserData(userId, requestingUserId):
    # Vulnerable: No authorization check
    return database.query(f"SELECT * FROM users WHERE id = {userId}")
```

A02 - Cryptographic Failures

```
def hashPassword(password):
    # Vulnerable: Weak hashing, no salt
    return hashlib.md5(password.encode()).hexdigest()
```

A03 – Injection

```
def findUserByEmail(email):
    # Vulnerable: SQL injection
    query = f"SELECT * FROM users WHERE email = '{email}'"
    return database.execute(query)
```


© 2025 QA Limited or its affiliates. All rights reserved.

A07 - Identification and Authentication Failures

```
def resetPassword(email, newPassword, resetToken):
    # Vulnerable: No token validation, weak session management
    user = findUserByEmail(email)
    user.password = hashPassword(newPassword)
    return "Password reset successful"
```

Phase 3: Security Analysis Exercise

1. Manual Code Review: Use security-focused checklist to identify vulnerabilities
2. Tool-Assisted Analysis: Run SonarQube, Bandit, or similar security scanners
3. Compare Results: Manual detection vs automated tool detection

Phase 4: Secure Implementation

Fix identified vulnerabilities with proper implementations:

- Parameterized queries for injection prevention
- Strong password hashing (bcrypt/Argon2)
- Proper authorization checks
- Secure token generation and validation
- Input validation and sanitization
- Rate limiting for authentication attempts

Extension

Phase 5: Security Testing

Create test cases that attempt to exploit the original vulnerabilities:

- SQL injection payloads
- Authorization bypass attempts
- Brute force attack simulations
- Token manipulation tests

© 2025 QA Limited or its affiliates. All rights reserved.

Success Criteria:

- All OWASP Top 10 vulnerabilities identified and fixed
- Security tools show zero critical/high issues
- Penetration testing scenarios pass
- Performance remains acceptable with security measures

Output: Secure authentication system with security test suite and vulnerability assessment report

© 2025 QA Limited or its affiliates. All rights reserved.